
AI B Project: Cartpole

Wang Shinan
Department of Artificial Intelligence
12411607@mail.sustech.edu.cn

Zheng Renjie
Department of Artificial Intelligence
12412840@mail.sustech.edu.cn

Yang Yiming
Department of Statistics and Data Science
12411332@mail.sustech.edu.cn

Abstract

We explored various RL algorithms to solve the CartPole-v1 task in both online and offline settings. We implemented and compared several online methods, including DQN, PPO, A2C, and Rainbow, finding that Rainbow offered the highest efficiency and stability. For offline learning, we utilized BC, BCQ, and a simplified World Model to replicate or improve upon expert policies. Experimental results demonstrate that while traditional methods like DQN exhibit instability, integrated approaches and proper hyperparameter and loss tuning significantly enhance agent performance and convergence speed.

1 Online

1.1 DQN

We switched to soft target-network updates with $\tau = 0.01$ for smoother learning. And we set epsilon-decay only happen after 2000 steps to avoid losing exploration ability. Some other hyper-parameter changes are shown in Table 1. We notice DQN is very unstable and has great performance drop in long term(20). This would be discussed further in 1.4.

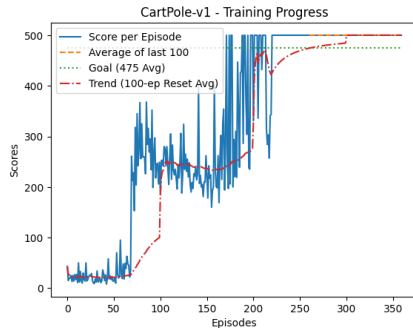


Figure 1: DQN training curve

Hyper-parameters	Value
hidden size	64
batch size	128
memory size	50000

Table 1: DQN hyperparameters

1.2 PPO

1.2.1 Experiments

We trained the model(hyperparameters shown in 2) using a batch size of 64, a rollout length of 128, a maximum gradient norm of 0.5, and 15 training epochs with Adam optimizer.

The model achieved an average score of 500.0 in Evaluation 100 rounds.

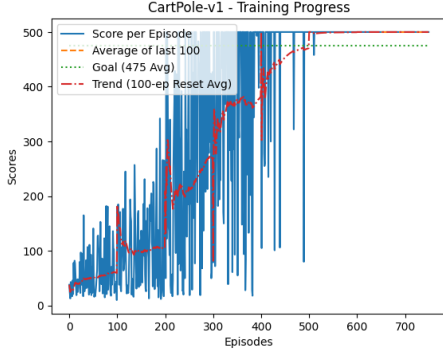


Figure 2: PPO Score

Hyper-parameters	Value
hidden size	64
GAE Lambda	0.95
LR(Linear decending)	1e-3 $\rightarrow$ 1e-4
CLIP EPSILON	0.2
ENTROPY COEF	0.01
VALUE COEF	0.5

Table 2: Hyper-parameters

1.2.2 Influences of hyper-parameters

1. rollout length

(a) **256:**

As shown in 11, the convergence speed barely changes, but the maximum score decreases and the stability after convergence decreased. This is because, when the rollout length increases, each update uses more data and the overall update frequency becomes lower. In short-episode tasks such as CartPole, a longer rollout may mix trajectories from multiple episodes or policies together, which in turn increases instability.

(b) **32:**

As shown in 12, the overall trend is similar to that with 128, but the training process is more unstable and exhibits greater variance. This is because, when the rollout length is shorter, each update is more affected by randomness in the data, leading to larger variance.

2. EPS(Epochs Per Sample): the number of times each sample is trained

$$\text{EPS} = \text{epochs} \times \frac{\text{rollout-length}}{\text{minibatch-size}} \quad (1)$$

Schulman et al. point out in Appendix B of PPO paper that the number of epochs (the times sampled data are repeatedly iterated) is the most important hyperparameter of PPO Schulman et al. [2017].

Fixing the **rollout** value as 128, keep other hyper-parameters and training parameters(train for 700 episodes) the same, we tested the following parameter combinations and the corresponding training performance with different EPS values(see Appendix Figures 13 to 19).

PPO Epochs	Batch size	EPS	last 100 average score
1	128	1	41.19(not converged)
3	128	3	107.32(not converged)
6	128	6	130.46(not converged)
6	64	12	197.43
12	64	24	433.20

Table 3: EPS value and corresponding performance

From the chart and figures, it can be seen that as EPS increases, the training performance improves, but the stability decreases rapidly. This is because

- When EPS is too large, a single sample is trained so many times that r_t is out of $[1 - \epsilon, 1 + \epsilon]$ and is clipped, making the training noise amplifies.

PPO Epochs	Batch size	EPS	last 100 average score
6	64	12	197.43
12	128	12	250.22
24	256	12	500.00

Table 4: fixed EPS

- When EPS is too small, though r_t will not be clipped often, the learning signal is too weak, leading to slow convergence.

From the chart and figures, it could be seen that when EPS is fixed, the trend and oscillations of the graphs are similar; while as **batchsize** and **epochs** increases the final performance is better. This is because a larger **batchsize** provides a lower-variance estimate of the policy gradient, making the estimation of the advantage function and value loss more stable, while also reducing bootstrap noise.

1.3 A2C

1.3.1 Parameter Analysis

During the A2C training process, hyperparameter settings have a significant impact on convergence performance. Table 5 lists the key hyperparameters of the algorithm.

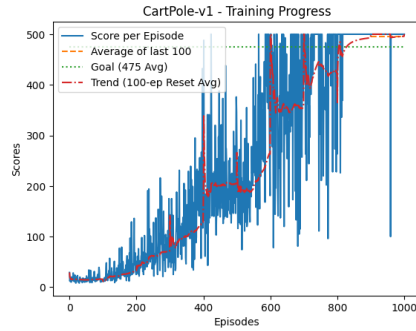


Figure 3: A2C Score

Parameter	Value
Gamma (γ)	0.99
Hidden Dimension	128
Actor Learning Rate	1×10^{-3}
Critic Learning Rate	1×10^{-2}
Entropy Coef	1×10^{-3}
Optimizer	Adam

Table 5: A2C Hyperparameter

Differential Learning Rate Settings Experiments showed that setting the Critic’s learning rate to 10 times that of the Actor (Critic LR = 0.01, Actor LR = 0.001) significantly improves convergence speed. Meanwhile, if the Actor’s learning rate is higher than the Critic’s, the Agent learns almost nothing (see Fig 28 to 29). Furthermore, if both learning rates are identical or too low, it is difficult to achieve good results within a limited number of steps (see Fig 30 and 31).

- **Analysis:** The Critic provides the "Baseline" required for the Actor’s updates. If the Critic converges too slowly, the Actor will update based on incorrect value estimates for a long time, causing the policy to oscillate around suboptimal solutions. A higher Critic learning rate ensures that value estimation quickly keeps up with policy changes, thereby providing accurate advantage signals δ_t .

It was also found that using parameters 1e-2 and 1e-1 can also achieve rapid convergence, but significant oscillations may occur during training. Although convergence to suboptimal solutions was not observed in the experiments, (see Figure 32).

1.3.2 Entropy Regularization

Experiments showed that adopting entropy regularization helps the Agent avoid converging to suboptimal solutions, makes the entire training process smoother, and leads to faster convergence (see Figure 33).

- **Principle of Entropy Regularization:** Add a term for the entropy of the policy distribution to the Actor's loss

$$H(\pi(s)) = - \sum \pi(a|s) \log \pi(a|s)$$

$$L_{actor} = -\mathbb{E}[\log \pi(a|s)A(s, a)] - \beta \cdot H(\pi(s))$$

Effect: The larger the entropy, the more uniform the distribution (closer to random). Maximizing entropy encourages the Agent to try different actions during the early stages of training and prevents concentrating probability on a single action too early.

1.4 Rainbow

1.4.1 How we find rainbow

Initially, we want to train DDQN, however during training, we find some unexpected results which made it hard to explain in the training progress below, which is called "catastrophic forgetting", and in the paper Roderick et al. [2017], they explain several reasons: "One of the reasons this forgetting occurs is the inherent instability of approximating the Q-function over a large state-space using these Bellman updates.... These additions, and others, to the DQN algorithm improve its stability significantly, but the network still experiences catastrophic forgetting. ", "Another reason this catastrophic forgetting occurs is that the algorithm is learning a proxy, the Q-values, for a policy instead of approximating the policy directly." So, theoretically, we could not solve catastrophic forgetting, because of Function Approximation, Bootstrapping and Off-policy. But we find a rainbow algorithm which combines advantages from different variants of DQN to greatly alleviate catastrophic forgetting.

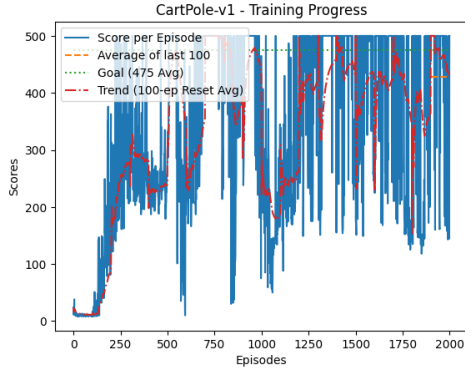


Figure 4: DDQN Score

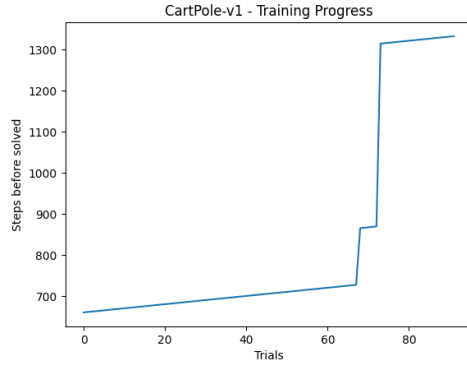


Figure 5: DDQN Solved

1.4.2 Algorithm Introduction

Rainbow is not a brand-new algorithm, but rather a systematic integration by DeepMind Hessel et al. [2017] of six of the most effective improvements developed in recent years, addressing the many limitations of the DQN algorithm (such as low sampling efficiency, blind exploration strategies, and Q-value overestimation). The implementation in this paper includes the following core components:

Noisy Nets Noisy Nets replace ϵ -greedy by injecting learnable noise into weights for state-dependent exploration:

$$y = (\mu^w + \sigma^w \odot \epsilon^w)x + (\mu^b + \sigma^b \odot \epsilon^b) \quad (2)$$

The parameter σ automatically regulates noise levels, enabling a natural transition from exploration to exploitation during training.

Distributional RL C51 models the full return distribution instead of expected scalars to capture multimodality. It discretizes values into $N_{atoms} = 51$ fixed supports:

$$Z_\theta(s, a) = \sum_{i=1}^{N_{atoms}} p_i(s, a) \delta_{z_i} \quad (3)$$

Learning the complete distribution enhances feature robustness and convergence speed.

Multi-step Learning N-step TD ($n = 3$) is used to accelerate reward propagation and balance the bias-variance trade-off. The target is calculated as:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} q_{\theta}(s_{t+n}, a') \quad (4)$$

This facilitates faster credit assignment compared to traditional 1-step TD.

Other Components We also integrate **Double Q-learning** to mitigate maximization bias, **Dueling Networks** to decouple $V(s)$ and $A(s, a)$, and **Prioritized Experience Replay (PER)**. PER prioritizes learning from high-error samples by setting sampling probabilities $P(i) \propto |\delta_i|^\omega$ based on TD-error (or KL divergence).

1.4.3 Experimental Results

The parameters used in the experiment are shown in the table above. As shown in Figure 6, the Rainbow algorithm demonstrates extremely fast convergence speed. Thanks to the introduction of N-step and Distributional RL, the algorithm’s information utilization rate is extremely high in the early stages, and Noisy Nets ensure sufficient exploration without sacrificing performance.

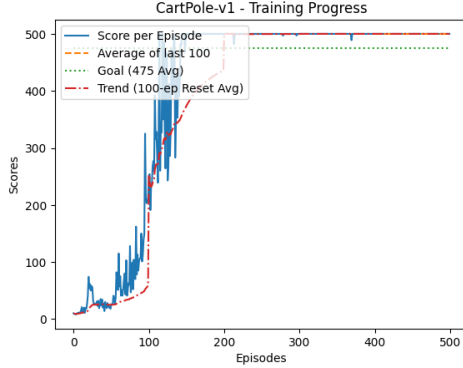


Figure 6: Rainbow Score

Parameter	Value
Gamma (γ)	0.99
Batch Size	32
Learning Rate	1×10^{-4}
Multi-step (n)	3
Atom Size (C51)	51
Value Range ($V_{\min}, V_{\max}$)	$-10, 500$
Noisy Net σ_{init}	0.5
PER α	0.6
Optimizer	Adam

Table 6: Rainbow Key Hyperparameter Settings

Compared to DDQN, Rainbow not only converges faster but also has smaller variance in the later stages of training. No obvious performance collapse occurred, proving the robustness of the combined improvement mechanisms.

1.4.4 Ablation Studies

In order to verify the contribution of each component in the Rainbow algorithm, we conducted ablation studies by adjusting key hyperparameters.

- **Prioritized Experience Replay parameter α :**
We changed the prioritization exponent from $\alpha = 0.6 \rightarrow \alpha = 0.0$ (see Fig. 34). Setting $\alpha = 0$ degrades the mechanism to uniform sampling. In CartPole, "failure" transitions are rare but contain high information value (high TD error). Without prioritization, the agent replays these critical samples less frequently, resulting in training instability and lower sample efficiency.
- **Multi-step TD parameter n :**
We adjusted the parameter from $n_{\text{step}} = 3 \rightarrow n_{\text{step}} = 1$ (see Fig. 35). Because CartPole is a task with sparse failure signals and continuous survival rewards, the multi-step return mechanism helps propagate the reward signal to earlier states more efficiently. Reverting to standard 1-step TD significantly slows down the reward propagation, leading to slower convergence speed compared to the baseline.
- **Noisy Nets parameter σ_{init} :**
We reduced the initial noise standard deviation from $\sigma_{\text{init}} = 0.5 \rightarrow \sigma_{\text{init}} = 0.01$ (see Fig.

36). Since our implementation removed the traditional ϵ -greedy strategy, relying entirely on parametric noise for exploration, setting $\sigma_{init} = 0.01$ makes the policy nearly deterministic. This lack of exploration prevents the agent from discovering optimal strategies, may causing it to get stuck in local optima or fail to learn entirely.

- **Distributional Atom Size N_{atoms} :**

We decreased the granularity of the value distribution from $atom_size = 51 \rightarrow atom_size = 11$ (see Fig. 37). A higher number of atoms ($N = 51$) allows the network to model the multimodal nature of returns with higher precision, which implicitly acts as an auxiliary task to learn better state representations. Reducing the number of atoms leads to a coarser approximation of the value function and increased variance in the training curve.

2 Offline

2.1 Behavior Copy

We run the trained PPO model for 100 episodes, obtaining $100 \times 500 = 50000$ state-action pairs. Treating the mapping from *state* to *action* as a classification task, we employ a simple MLP on this data.

2.1.1 MLP structure

Let the hidden layer dimension (hidden dim) be d , utilizing a 3-layer MLP : $\text{Linear}(4 \times d) \rightarrow \text{ReLU} \rightarrow \text{Linear}(d \times d) \rightarrow \text{ReLU} \rightarrow \text{Linear}(d \times 2)$

We train the MLP for 100 epochs with a batch size of 64, using the Adam optimizer with a linearly decaying learning rate scheduler.

2.1.2 Performance

Picking $d = 128$, the loss curve is shown in 10. The average score of 100 evaluating is 500.0 .

2.1.3 Ablation Studies

We further tested the model with hidden size $d = 64, 32, 8, 4, 2, 1$, the corresponding loss curve and evaluation score are shown in 27.

As shown in the figure, even when the hidden size is reduced to 4, the model can still achieve a score of 500.0. A possible reason is that the state and action spaces of cartpole are only 4 and 2 dimensions respectively, making it a very simple classification problem, hence a small MLP can achieve very good performance.

2.2 BCQ

BC would suffer from OOD issues when the environment changes slightly (e.g., different start states), due to distributional shift. BCQ (Fujimoto et al. [2019]) mitigates this issue by constraining action selection to remain close to the behavior policy distribution represented in the offline dataset.

2.2.1 Implementation Details

We train a supervised behavioral model G_ω to approximate the dataset policy $P(a \mid s)$. During evaluation and Q-learning, we discard any action whose probability ratio

$$\frac{G_\omega(a \mid s)}{\max_{\hat{a}} G_\omega(\hat{a} \mid s)} < \tau$$

falls below the threshold τ . The Q-learning target is then computed only over these allowed actions to avoid OOD overestimation.

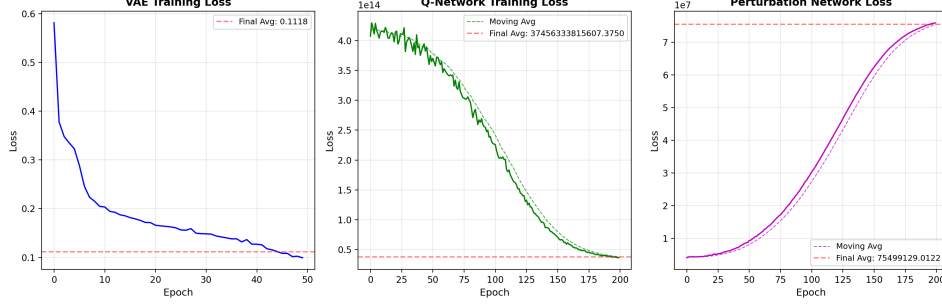


Figure 7: BCQ score

2.2.2 Experimental Results

We train BCQ using the same 50,000 PPO transitions collected in BC, with $\tau = 0.3$, learning rate 1×10^{-3} , batch size 64, hidden dimension 128, and a target update frequency of 500 steps.

As shown in Figure 7, BCQ trains much more stably than Offline-DQN, avoiding value divergence and reaching a consistent score of 500.

2.2.3 Ablation: Impact of Threshold τ

We tested the impact of threshold τ . A high threshold (e.g., $\tau = 0.9$) forces the agent to closely follow the behavior policy, while a low threshold (e.g., $\tau = 0.01$) allows OOD actions and risks value overestimation. In CartPole, $\tau = 0.3$ achieves the best balance between stability and improvement.

2.3 Others: World Model(MLP + CEM)

Inspired by Ha and Schmidhuber [2018], we collect a dataset generated entirely from random interactions, and train a lightweight MLP as the **world model** (the original work employs a RNN, we simplify it to an MLP due to the simplicity of the CartPole environment). The model predicts the next state given the current state and action, i.e., $\hat{s}_{t+1} = f_{\phi}(s_t, a_t)$.

Controller (C-model / policy model) is optimized using the Cross-Entropy Method (CEM) over the policy parameter distribution. The procedure is as follows:

1. Repeatedly sample a batch of policy parameters θ from a Gaussian distribution $\mathcal{N}(\mu, \sigma)$;
2. For each sampled policy, perform rollouts under the learned world model and compute the cumulative reward;
3. Select the top- k policies with the highest returns as elite samples;
4. Update the distribution parameters (μ, σ) using the elite set.

Notably, training **Controller** does not require any interaction with the real environment.

2.3.1 Training

Directly applying MSE loss on predicting next state has a really bad performance, along with serious OOD problem (the average score is around 150). As a result, the training procedure requires following modifications:

validation set We use 10% of the data for validation and select the model with the lowest validation loss as best model.

Normalization The cartpole environment is highly sensitive to state changes. Applying MSE loss directly would lead to tiny loss and low sensitivity to details.

Hence we normalize both the input and output states of the model/data as follows:

$$\tilde{s} = \frac{s - \mu}{\sigma + \epsilon} \quad (5)$$

where s denotes the original state, $\tilde{s}$ the normalized state, μ, σ are the mean and standard deviation of the training set.

Autoregressive Prediction Predicting only next state gives modal a relatively weak constraint (in practice, the loss drops below $1e-5$ in only 15 epochs), hence we train the model using an autoregressive objective that predicts future states for $H = 10$ steps, encouraging a better long-term modeling. The final loss function is as follows:

$$L = \frac{1}{H} \sum_{k=1}^H \|\hat{s}_{t+k} - \tilde{s}_{t+k}\|_2^2 \quad (6)$$

2.4 Results and Analysis

We randomly sample 1,000 episodes (resulting in 13,499 transitions) to train the world model M (MLP, hidden size 64); the loss curve is shown in 8. Using this model, we train a controller C (MLP with hidden size 8) with CEM (60 iterations, population 128, elite fraction 0.1). When evaluated in the real environment over 100 episodes, the policy achieves an average score of 500.0. We further tested with different random seeds for 10 times, all resulting in 500.0.

Notably, Doubling the hidden size of M yields an average score of 499.73 over 100 evaluation episodes, with occasional drops to 485, suggesting possible overfitting and OOD effects. This phenomenon aligns with known challenges of deploying world models under real-world distribution shifts.

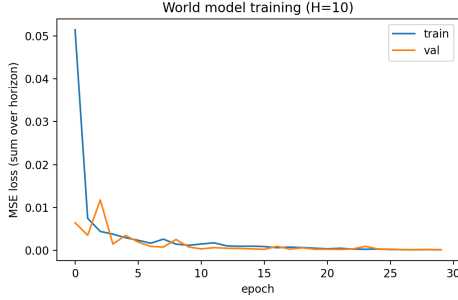


Figure 8: loss curve of M model(trained for 200epoch, flat tail omitted)

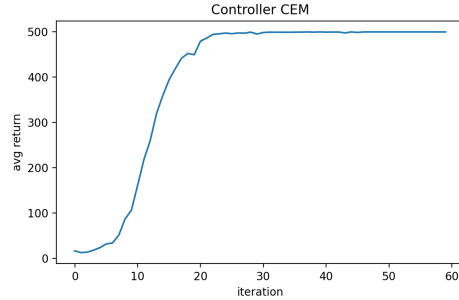


Figure 9: training curve of model C

3 Conclusion

We estimated the convergence steps of above methods (using their best-performing hyperparameter) by visually inspecting their trend(100-ep Reset Avg), and summarized the results in table 7 and 8.

Method	approximate steps to converge
DQN	300(drop at 600)
PPO	500
A2C	800
Rainbow	200

Table 7: Converge speed comparison

Method	approximate steps to converge
BC	15 epochs
BCQ	200 epochs
WM (M)	10 epochs
WM (C)	20 epochs

Table 8: Converge speed comparison (Offline)

As shown in Table 7 and 8, Rainbow is the most efficient and stable online algorithm for this type of control task, effectively addressing the instability issues of traditional DQN. In offline scenarios, both Behavior Cloning (BC) and World Models exhibit very high efficiency; in particular, BC is able to perfectly reproduce the expert policy using a highly simplified network architecture.

References

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. Appendix B.
- Melrose Roderick, James MacGlashan, and Stefanie Tellex. Implementing the deep q-network, 2017. URL <https://arxiv.org/abs/1711.07478>.
- Matteo Hessel, Joseph Modayil, H. V. Hasselt, T. Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, M. G. Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. pages 3215–3222, 2017.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration, 2019. URL <https://arxiv.org/abs/1812.02900>.
- David R Ha and J. Schmidhuber. World models. *ArXiv*, abs/1803.10122, 2018.

A Appendix

A.1 Additional Images

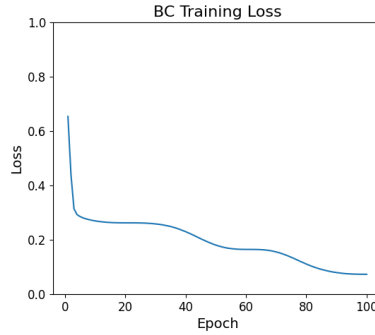


Figure 10: BC: MLP loss curve

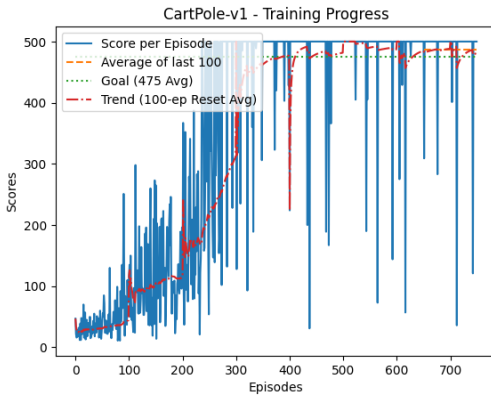


Figure 11: PPO, rollout-length = 256

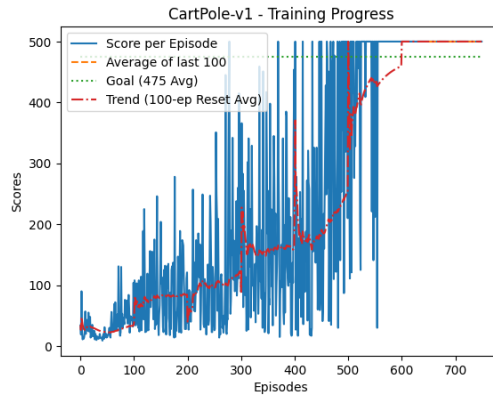


Figure 12: PPO, rollout-length = 64

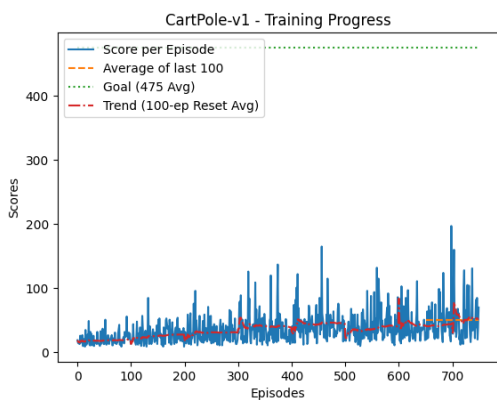


Figure 13: PPO eps = 1

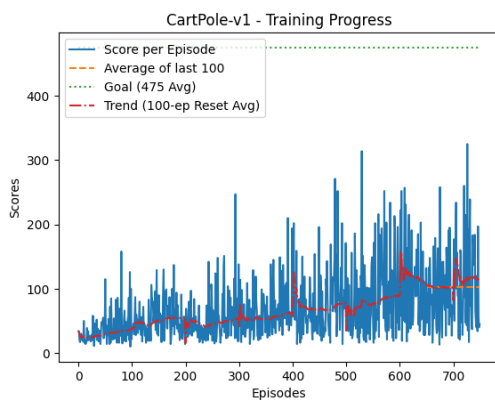


Figure 14: PPO eps = 3

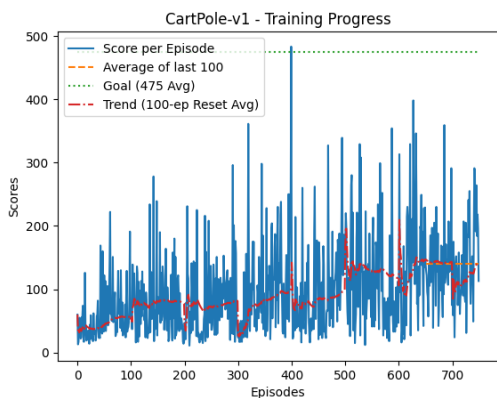


Figure 15: PPO eps = 6

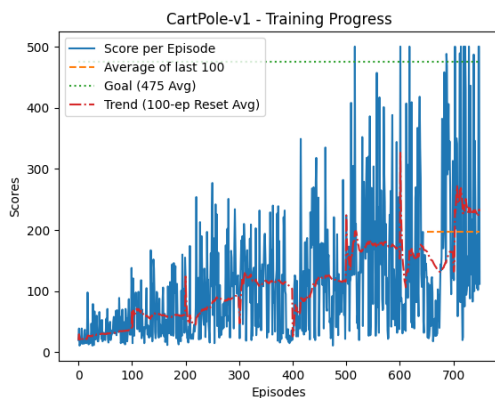


Figure 16: PPO eps = 12

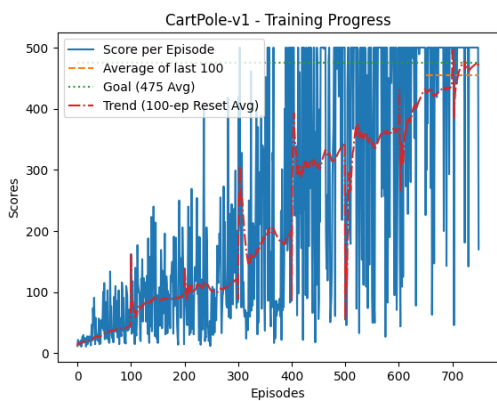


Figure 17: PPO eps = 24

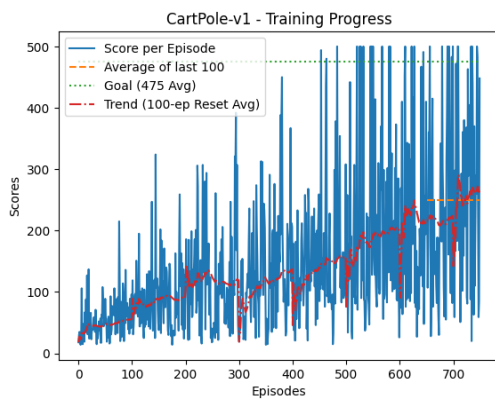


Figure 18: PPO eps = 12, experiment 2 of fixed EPS

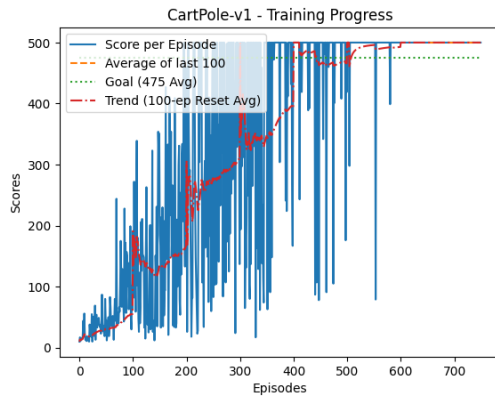


Figure 19: PPO eps=12, experiment 3 of fixed EPS

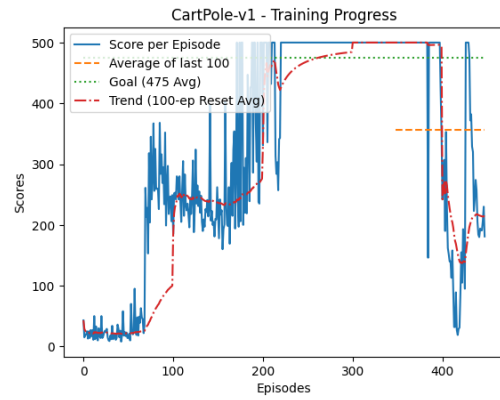


Figure 20: Bad result of DQN in long term

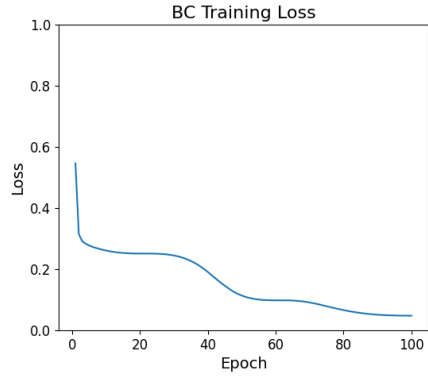


Figure 21: $d = 64$, avg_score = 500.0

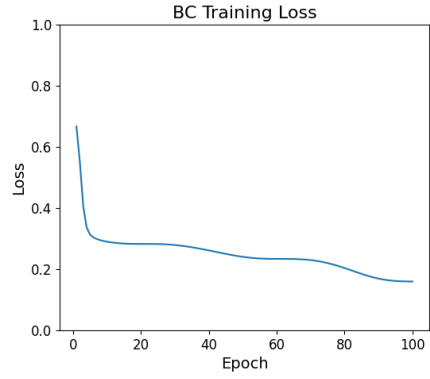


Figure 22: $d = 16$, avg_score = 500.0

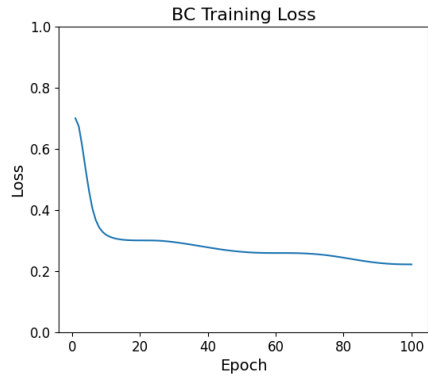


Figure 23: $d = 8$, avg_score = 500.0

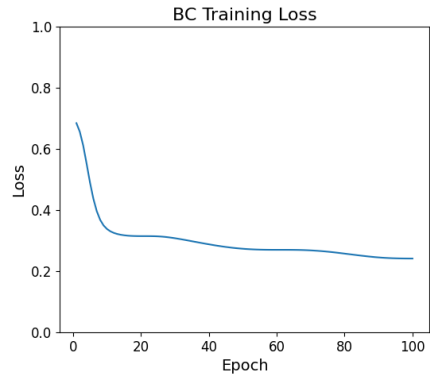


Figure 24: $d = 4$, avg_score = 500.0

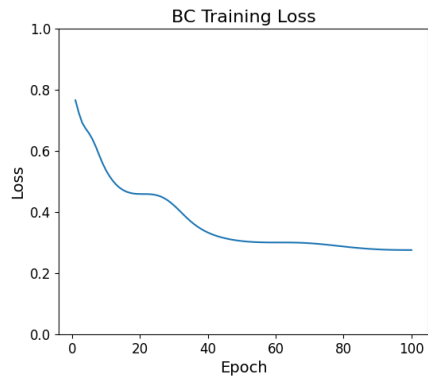


Figure 25: $d = 2$, avg_score = 469.90

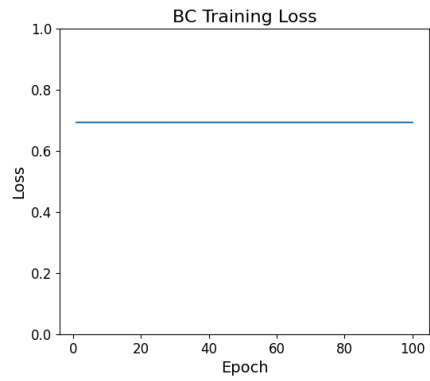


Figure 26: $d = 1$, avg_score = 9.36

Figure 27: BC ablation studies

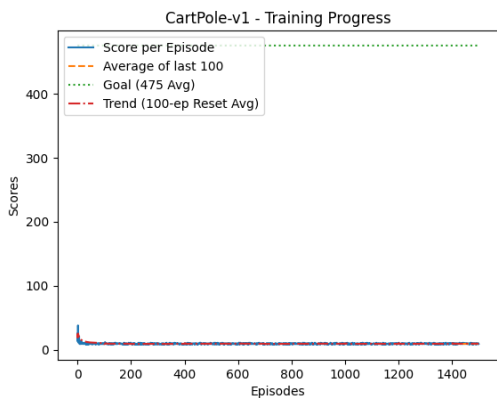


Figure 28: a2c: $1e-2$ $1e-4$

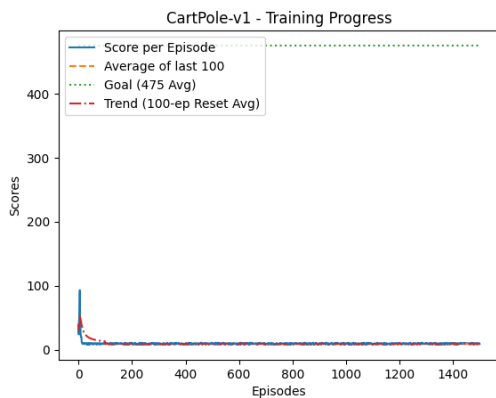


Figure 29: a2c: $1e-1$ $1e-2$

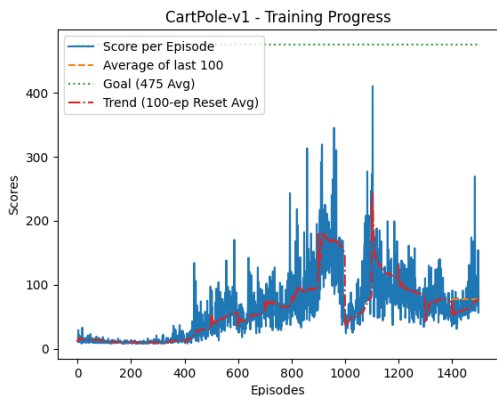


Figure 30: a2c: $1e-3$ $1e-3$

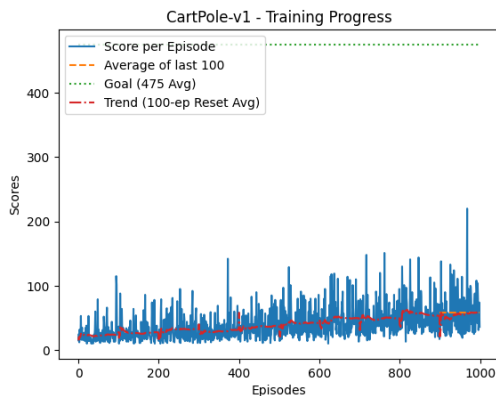


Figure 31: a2c: $1e-4$ $1e-2$

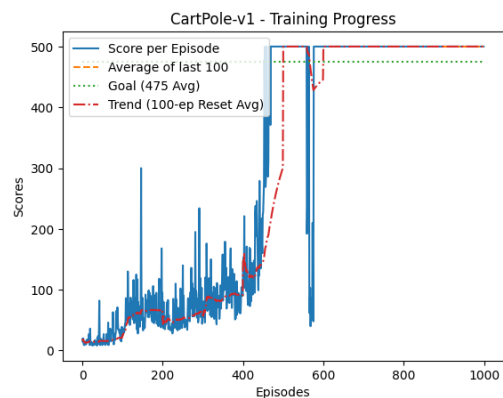


Figure 32: a2c: $1e-2$ $1e-1$

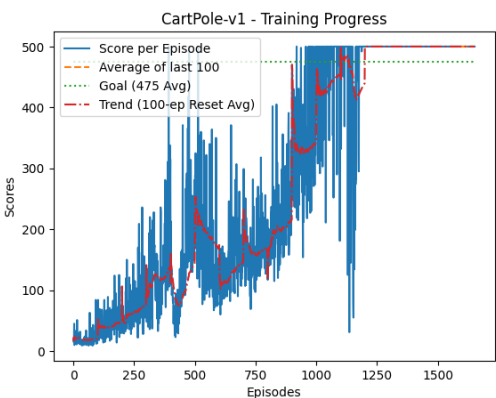


Figure 33: a2c: without entropy

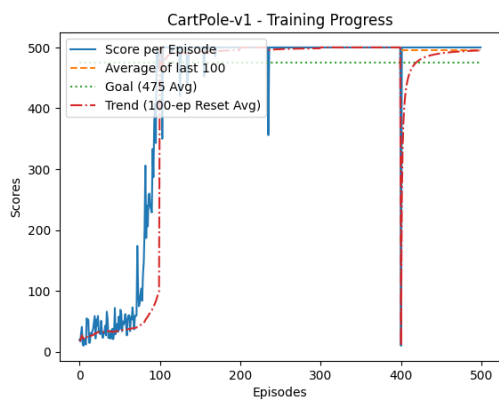


Figure 34: rainbow: $\alpha=0$

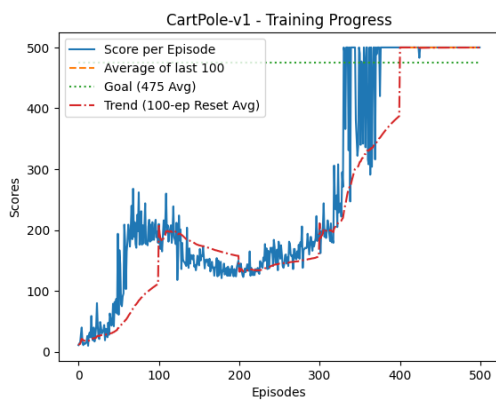


Figure 35: rainbow: $n=1$

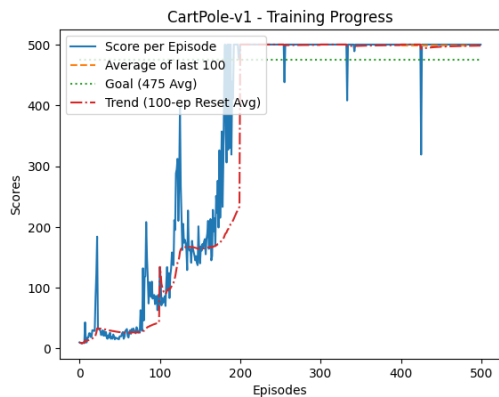


Figure 36: rainbow: $\sigma=0.01$

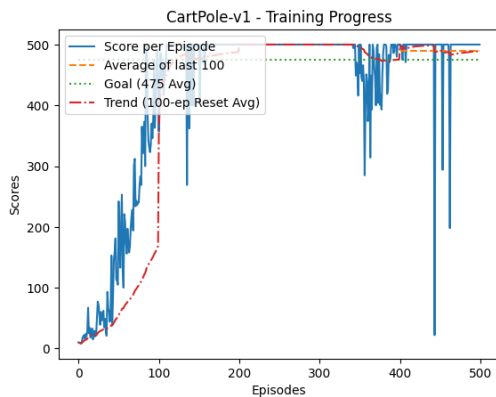


Figure 37: rainbow: $\text{atom}=11$